

CHAPTER 04

API Gateway

The single front door — deep dive

Routing

Authentication

Rate Limiting

Service Discovery

YARP / Nginx

What We'll Cover

01

Why a Gateway?

The problem with no single entry point

02

Routing

How requests get sent to the right service

03

Authentication

Validate JWT once — not in every service

04

Rate Limiting

Protect services from traffic abuse

05

Service Discovery

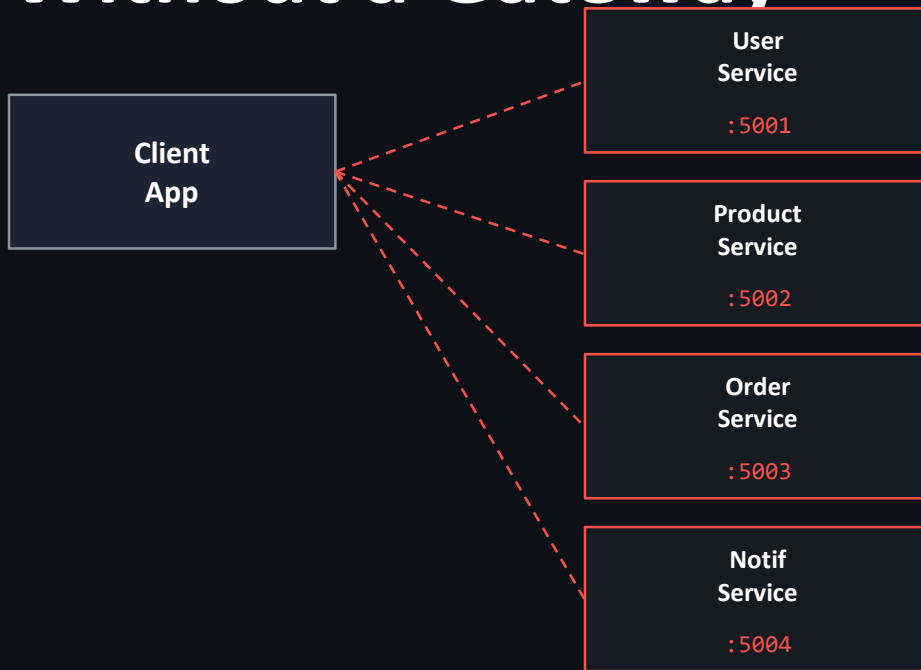
How the gateway finds services dynamically

06

YARP in Practice

Real config with C# / .NET

Without a Gateway



✗ Client must know every service address

✗ Auth logic duplicated in every service

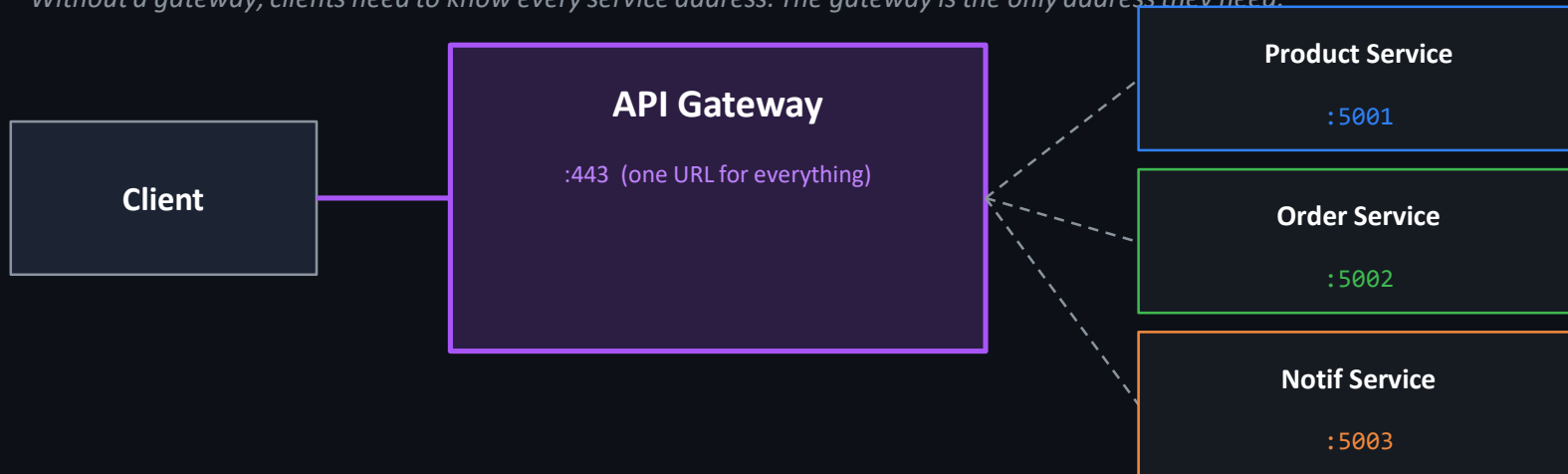
✗ No central rate limiting or logging

✗ CORS, SSL handled per service

💡 Solution: one front door that handles all of this centrally — the API Gateway

The Single Front Door

Without a gateway, clients need to know every service address. The gateway is the only address they need.



🔄 Routing

/products/* → ProductService
/orders/* → OrderService

🔒 Authentication

Validate JWT once here,
not in every service

⚡ Rate Limiting

Block clients sending
too many requests

Routing — Send Requests to the Right Service

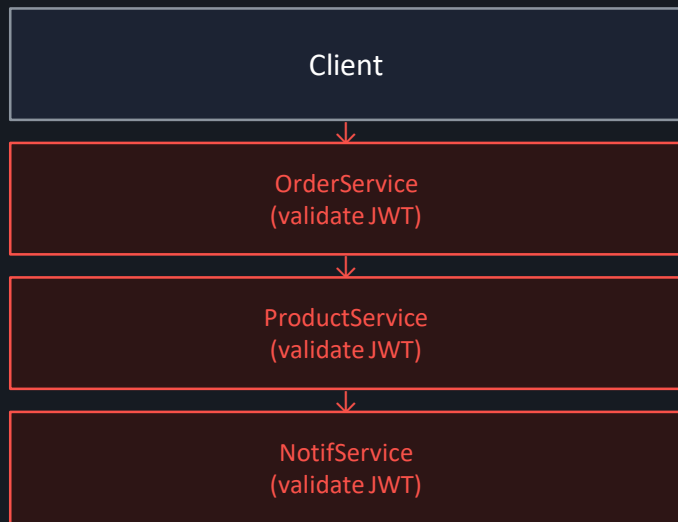
The gateway inspects the URL path and forwards the request to the correct upstream service.

Incoming Path	Routes to
/products/**	→ ProductService :5001
/orders/**	→ OrderService :5002
/payments/**	→ PaymentService :5003
/notifications/**	→ NotifService :5004

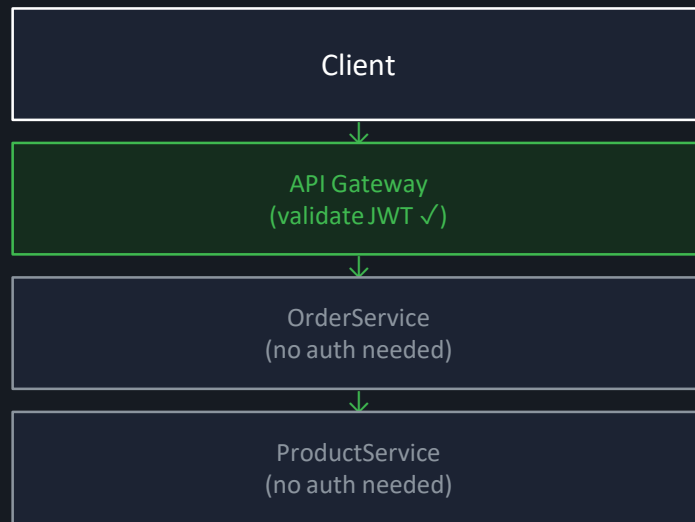
In YARP (C# / .NET): routes configured in appsettings.json — each route has a RouteId, ClusterId, and a PathMatch pattern.

Validate JWT Once — Not in Every Service

✗ Without Gateway Auth



✓ With Gateway Auth



💡 The gateway adds the user identity to the request headers — services trust that the gateway already verified it.

Rate Limiting — Protect Your Services

Without rate limiting, a single abusive client can take down your entire system.

Fixed Window

Allow N requests per time window (e.g. 100 req/min). Simple but can burst at window edges.

100 requests / 60 seconds

Sliding Window

Track requests over a rolling time period. Smoother than fixed window, no edge bursting.

100 requests in last 60s

Token Bucket

Tokens fill up over time. Each request consumes a token. Allows controlled bursts.

10 tokens/sec, max burst 50

What happens when the limit is exceeded?

HTTP 429 Too Many Requests

Gateway returns 429 immediately — the request never reaches your services. Include Retry-After header to tell clients when to try again.

How the Gateway Finds Services

In production, services don't have fixed addresses. The gateway needs a live map.



Service Registry

ProductService → 10.0.0.4:5001, 10.0.0.5:5001 (2 instances)

OrderService → 10.0.0.7:5002 (1 instance)

NotifService → 10.0.0.9:5003 (1 instance)

Consul

HashiCorp's service mesh & discovery

Kubernetes DNS

Built into K8s — automatic

AWS Cloud Map

Managed discovery on AWS

YARP in Practice

YARP (Yet Another Reverse Proxy) — Microsoft's official .NET API Gateway library.

appsettings.json

```
"ReverseProxy": {  
  "Routes": {  
    "products-route": {  
      "ClusterId": "products-cluster",  
      "Match": { "Path": "/products/{**catch-all}" }  
    },  
    "orders-route": {  
      "ClusterId": "orders-cluster",  
      "Match": { "Path": "/orders/{**catch-all}" }  
    }  
  },  
  "Clusters": {  
    "products-cluster": {  
      "Destinations": {  
        "dest1": { "Address": "http://localhost:5001/" }  
      }  
    }  
  }  
}
```

1

Install NuGet package

```
dotnet add package Yarp.ReverseProxy
```

2

Register in Program.cs

```
builder.Services.AddReverseProxy()  
    .LoadFromConfig(config.GetSection("ReverseProxy")  
    );
```

3

Map the middleware

```
app.MapReverseProxy();
```

Key Takeaways

01 Single Entry Point

One URL for all clients. Services stay hidden behind the gateway — clients never know service addresses.

02 Routing

URL path patterns map to the correct upstream service. Configure once, change without touching clients.

03 Auth at the Edge

Validate JWT once at the gateway. Services trust the gateway — no repeated auth logic.

04 Rate Limiting

Protect services from abuse. Return 429 at the gateway — upstream services never see the bad traffic.

05 Service Discovery

A service registry maps names to dynamic addresses. The gateway looks up services at runtime.

06 YARP in .NET

Simple config-driven reverse proxy. Install, configure routes & clusters, add middleware — done.